

# Assistant Agentic RAG en Informatique

Rapport — Projet de fin de module, Master IBDDC — Évaluation finale

Auteur : Adil Hammad

Formation : Master IBDDC

Date : 24 juin 2026

## 1. Démarche suivie

L'objectif était de construire un assistant **Agentic RAG** (Retrieval-Augmented Generation piloté par un agent) spécialisé dans le domaine de l'informatique : programmation, algorithmes, structures de données, systèmes, réseaux, bases de données, débogage et cybersécurité. La démarche s'est articulée en cinq étapes.

- **Constitution du corpus.** Téléchargement de guides publics au format PDF stockés dans `data/pdfs_informatique/`.
- **Indexation.** Découpage en chunks de 1000 caractères (chevauchement 200) via `RecursiveCharacterTextSplitter`, vectorisation et persistance dans Chroma (`ingest.py`).
- **Conception de l'agent.** Implémentation d'un graphe LangGraph (`StateGraph`, sans `create_agent`) suivant le pattern `retrieve → grade → generate / rewrite`.
- **Outils.** Définition de trois outils métier exposés au LLM pour cadrer et documenter chaque question.
- **Validation.** Tests unitaires sur le routage et les outils, puis protocole d'évaluation sur 20 questions (10 simples + 10 complexes).

## 2. Fonctionnement du système

### 2.1 Pile technique

| Composant                                  | Choix                                                              |
|--------------------------------------------|--------------------------------------------------------------------|
| LLM (raisonnement, grading, reformulation) | Ollama local — <code>llama3.2:3b</code> (temperature 0)            |
| Embeddings                                 | HuggingFace <code>all-MiniLM-L6-v2</code> (sentence-transformers)  |
| Base vectorielle                           | Chroma, persistée dans <code>data/chroma_db/</code>                |
| Orchestration                              | LangGraph ( <code>StateGraph</code> + <code>InMemorySaver</code> ) |

### 2.2 Le graphe agentique

Le cœur du système est un graphe à quatre nœuds. L'état partagé (`AgentState`) transporte l'historique des messages, le nombre d'appels LLM, le compteur de reformulations, les documents récupérés et le verdict de pertinence.

- `agent` : le LLM décide d'appeler un outil ou de répondre directement (routage `route_after_agent`).
- `tools` : exécute les outils demandés ; les erreurs sont capturées et renvoyées au LLM.
- `grade_documents` : le LLM juge si les documents récupérés sont pertinents (oui / non).
- `rewrite_query` : reformule la question si les docs ne sont pas pertinents (jusqu'à `MAX_REWRITES = 2`).

La mémoire conversationnelle est assurée par un `InMemorySaver` indexé par `thread_id` (une session = un fil).

### 2.3 Outils exposés au LLM

- `classify_technical_domain(query)` : identifie le domaine technique dominant par correspondance de mots-clés.
- `extract_technical_keywords(query)` : extrait les termes techniques présents dans la question.
- `retrieve_documents(query)` : recherche sémantique des 4 passages les plus proches dans Chroma, avec citation de la source et de la page.

## 3. Protocole et résultats de l'évaluation

Le module `evaluation/run_evaluation.py` rejoue automatiquement un jeu de **20 questions** couvrant l'ensemble des domaines : 10 questions simples (définitions) et 10 questions complexes (comparaison SQL/NoSQL, sécurisation d'API, choix d'architecture pour 1 million de requêtes/jour...).

### 3.1 Métriques collectées

Pour chaque question, le harnais enregistre dans `evaluation/results/results.csv` :

| Métrique               | Description                                  |
|------------------------|----------------------------------------------|
| <code>time_s</code>    | Temps de réponse de bout en bout (s)         |
| <code>llm_calls</code> | Nombre d'appels au LLM dans le graphe        |
| <code>rewrites</code>  | Nombre de reformulations déclenchées (0 à 2) |
| <code>sources</code>   | Documents/pages récupérés pour répondre      |
| <code>answer</code>    | Réponse générée (contrôle qualitatif)        |

### 3.2 Grille de lecture attendue

| Axe                       | Questions simples        | Questions complexes                  |
|---------------------------|--------------------------|--------------------------------------|
| Récupération documentaire | 1 passe suffisante       | Récupération souvent nécessaire      |
| Appels LLM                | Faible (réponse directe) | Plus élevé (grade + génération)      |
| Reformulations            | Rares (0)                | Possibles si docs hors-sujet (1 à 2) |
| Temps de réponse          | Plus court               | Plus long (étapes supplémentaires)   |

### 3.3 Validation par les tests

Trois modules de tests (`pytest`) garantissent le comportement déterministe du graphe indépendamment du LLM :

- **Routing** (`test_graph_routing.py`) : route vers `tools` sur tool-calls, vers `grade_documents` après récupération, vers `rewrite_query` si pertinence insuffisante.
- **Récupération** (`test_retrieve.py`) et **outils** (`test_tools.py`) : classification de domaine, extraction de mots-clés et formatage des sources.

## 4. Limites

---

- **Grading binaire et coûteux.** Réponse libre oui/non du LLM, sensible au phrasing et coûtant un appel LLM par récupération.
- **Reformulation plafonnée.** Après 2 reformulations infructueuses, l'agent répond malgré des documents potentiellement non pertinents.
- **Modèle local limité.** `llama3.2:3b` offre un raisonnement modeste ; la qualité des réponses complexes en dépend directement.
- **Corpus restreint.** Quatre guides généralistes sans garde-fou explicite contre l'hallucination sur les sujets hors couverture.
- **Mémoire volatile.** `InMemorySaver` ne persiste pas les sessions au-delà de l'exécution du processus.
- **Évaluation sans vérité-terrain.** Métriques opérationnelles uniquement ; pas de scoring automatique de l'exactitude des réponses.

## 5. Pistes d'amélioration

---

- **Grading structuré.** Forcer une sortie JSON (score 0-1) pour fiabiliser le jugement de pertinence.
- **Récupération hybride + reranking.** Combiner recherche dense et BM25, puis réordonner les passages pour améliorer le rappel.
- **Évaluation qualitative.** Ajouter un *LLM-as-judge* ou un jeu de réponses de référence pour mesurer l'exactitude.
- **Persistence.** Remplacer `InMemorySaver` par un checkpointer SQLite/Postgres.
- **Modèle plus capable.** Tester `llama3.1:8b` ou un modèle API pour les questions complexes.
- **Corpus enrichi.** Élargir la base documentaire et tracer la couverture par domaine technique.